

PROYECTO UD1 – ACCESO A DATOS

Gestor de Setas

Joseba Fernández Hinojosa

2026-2027

Índice

Índice.....	2
1. Introducción.....	3
2. Descripción de la aplicación.....	3
3. Estructura del proyecto.....	3
4. Clases serializables.....	4
5. Gestión de ficheros.....	4
6. Operaciones CRUD.....	5
7. Relación e integridad de los datos.....	5
8. Gestión de XML mediante XStream.....	5
9. Control de errores.....	5
10. Tecnologías utilizadas y justificación.....	5
11. Interfaz gráfica de usuario (GUI).....	6
12. Manual de usuario.....	6
13. Puesta en marcha.....	7
14. Git y control de versiones.....	7
15. Conclusiones.....	7
16. Glosario.....	8

1. Introducción

El proyecto consiste en el desarrollo de una aplicación Java para gestionar información relacionada con diferentes especies de setas y sus avistamientos. La aplicación permite almacenar, consultar y modificar información mediante ficheros binarios utilizando objetos serializables, además de permitir la exportación y lectura de los datos mediante ficheros XML.

El objetivo principal es aplicar los conocimientos adquiridos en la unidad de acceso a datos, utilizando ficheros relacionados entre sí, proporcionando operaciones CRUD, serialización de objetos, gestión mediante XML y una interfaz gráfica que facilite el uso de la aplicación.

2. Descripción de la aplicación

La aplicación se denomina Gestor de Setas. Está formada por tres tipos principales de datos: especies, hábitats y avistamientos. Además de la versión mediante consola, se ha incorporado una interfaz gráfica básica para facilitar el uso de las principales operaciones.

Una especie contiene información como su identificador, nombre común, nombre científico, comestibilidad y descripción. Un hábitat contiene su identificador, nombre y descripción. Finalmente, un avistamiento registra una especie observada en un determinado hábitat, junto con la fecha, localización y observaciones.

Los avistamientos relacionan los otros dos tipos de información mediante los identificadores de especie y hábitat. De esta forma, los tres ficheros forman un conjunto de datos relacionado.

3. Estructura del proyecto

La estructura principal del proyecto es la siguiente:

ProyectoSetas/

```
|— src/
|   |— modelo/
|   |   |— Especie.java
|   |   |— Habitat.java
|   |   └─ Avistamiento.java
|   |— ficheros/
|   |   |— especie/
|   |   |— habitat/
|   |   └─ avistamiento/
|   |   └─ Utilidades.java
|   |— xml/
|   └─ principal/
|       └─ Main.java
```

```
|— datos/
|   |— Especies.dat
|   |— Habitats.dat
|   |— Avistamientos.dat
|   |— Especies.xml
|   |— Habitats.xml
|   |— Avistamientos.xml
|— lib/
|— Dependencias de XStream
```

4. Clases serializables

La aplicación utiliza tres clases que implementan la interfaz `Serializable`. Esto permite guardar directamente objetos Java en los ficheros binarios mediante `ObjectOutputStream` y recuperarlos posteriormente mediante `ObjectInputStream`.

4.1. Especie

La clase `Especie` representa una especie de seta. Sus atributos son: `id`, `nombreComun`, `nombreCientifico`, `comestibilidad` y `descripcion`. Dispone de constructor, getters, setters y método `toString()`.

4.2. Habitat

La clase `Habitat` representa el entorno donde puede encontrarse una seta. Contiene los atributos `id`, `nombre` y `descripcion`, además de constructor, getters, setters y `toString()`.

4.3. Avistamiento

La clase `Avistamiento` representa el registro de una observación. Contiene `id`, `idEspecie`, `idHabitat`, `fecha`, `localizacion` y `observaciones`. Los identificadores de especie y hábitat permiten relacionar el avistamiento con los otros ficheros.

5. Gestión de ficheros

La aplicación utiliza tres ficheros binarios diferentes: `Especies.dat`, `Habitats.dat` y `Avistamientos.dat`. Los objetos se almacenan mediante serialización.

Para escribir objetos se utilizan `FileOutputStream` y `ObjectOutputStream`. Para leerlos se utilizan `FileInputStream` y `ObjectInputStream`. Los registros se recorren secuencialmente hasta alcanzar el final del fichero, controlado mediante `EOFException`.

Para las operaciones de modificación y eliminación se utiliza un fichero auxiliar. Se lee el fichero original registro a registro y se escriben en el auxiliar los objetos que deben conservarse, aplicando las modificaciones cuando corresponde. Finalmente se sustituye el fichero original por el auxiliar.

Para las altas se utiliza escritura en modo append, evitando sobrescribir los registros existentes. Se controla también la cabecera del ObjectOutputStream para poder añadir objetos al fichero serializado.

6. Operaciones CRUD

La aplicación proporciona las operaciones principales de gestión de registros para las tres entidades.

Listar: muestra todos los registros almacenados.

Añadir: permite introducir un nuevo registro y comprueba que su identificador no exista.

Modificar: localiza un registro por su identificador y actualiza sus datos.

Eliminar: elimina un registro mediante el uso de un fichero auxiliar.

Buscar: localiza y muestra un registro a partir de su identificador.

7. Relación e integridad de los datos

Los tres ficheros no son independientes. Los registros de Avistamiento contienen los identificadores de la especie y del hábitat al que pertenecen.

Al añadir o modificar un avistamiento se comprueba que la especie y el hábitat indicados existan previamente. También se comprueba que no exista otro avistamiento con el mismo ID.

Para evitar inconsistencias, no se permite eliminar una especie si existe algún avistamiento que la utilice. La misma comprobación se realiza para los hábitats. De esta forma no quedan avistamientos apuntando a registros inexistentes.

8. Gestión de XML mediante XStream

La aplicación utiliza la librería XStream para convertir los objetos Java almacenados en los ficheros binarios a formato XML.

La opción de gestión XML permite exportar las especies, hábitats y avistamientos a tres ficheros XML independientes. Para realizar la exportación se leen primero los objetos de los ficheros binarios y se almacenan temporalmente en listas. Posteriormente XStream genera el contenido XML.

También se ha implementado la lectura de los XML mediante XStream. Los datos del XML se transforman de nuevo en objetos Java y se muestran por pantalla.

9. Control de errores

La aplicación incorpora control de errores básico para evitar que las operaciones con ficheros finalicen inesperadamente. Se controlan excepciones relacionadas con la entrada y salida de datos y con la lectura de objetos serializados.

También se comprueba la existencia de los ficheros antes de realizar determinadas operaciones y se muestran mensajes al usuario cuando no existe un registro, cuando se intenta repetir un identificador o cuando una operación no puede realizarse debido a las relaciones entre los datos.

10. Tecnologías utilizadas y justificación

Java Swing: se utiliza para desarrollar la interfaz gráfica de usuario. Permite crear ventanas, botones, formularios, tablas y cuadros de diálogo sin incorporar una tecnología externa al proyecto.

Java e IntelliJ IDEA se han utilizado como entorno principal de desarrollo. Java proporciona las clases necesarias para trabajar con ficheros, serialización y entrada de datos por consola. Para la interfaz gráfica se ha utilizado Java Swing.

La serialización mediante `ObjectOutputStream` y `ObjectInputStream` resulta adecuada para este proyecto porque permite almacenar directamente las instancias de las clases creadas. El acceso utilizado es secuencial, recorriendo los registros hasta encontrar el elemento buscado o alcanzar el final del fichero. Para una colección de tamaño reducido como la utilizada en el proyecto, este método es sencillo y suficiente.

El uso de ficheros auxiliares para modificar y eliminar registros permite reconstruir el fichero manteniendo los objetos que no deben eliminarse y aplicando las modificaciones sobre el registro correspondiente.

XStream se utiliza porque facilita la conversión entre objetos Java y documentos XML, permitiendo cumplir el requisito de exportación XML sin tener que construir manualmente todos los elementos del documento.

11. Interfaz gráfica de usuario (GUI)

Como mejora adicional, se ha incorporado una interfaz gráfica desarrollada con Java Swing. La GUI permite utilizar las funciones principales del gestor mediante ventanas, botones, formularios, tablas y cuadros de diálogo, ofreciendo una alternativa más cómoda al uso de la consola.

La ventana principal permite acceder a la gestión de especies, hábitats, avistamientos y operaciones XML. La interfaz reutiliza las clases existentes del proyecto para realizar las operaciones sobre los ficheros, evitando duplicar la lógica de acceso a datos.

En la gestión de especies, hábitats y avistamientos se pueden realizar las operaciones de listado, alta, búsqueda, modificación y eliminación. Los formularios permiten introducir los datos y las tablas facilitan la visualización de los registros.

También se mantienen las validaciones existentes, como la comprobación de identificadores duplicados, la existencia de especies y hábitats relacionados y las restricciones para eliminar datos que estén siendo utilizados por avistamientos.

La GUI utiliza cuadros de diálogo para mostrar errores, confirmaciones y resultados de las operaciones. De esta forma, la interfaz actúa como una capa de presentación sobre la lógica de gestión de ficheros.

12. Manual de usuario

Interfaz gráfica: desde la ventana principal se puede acceder a las secciones de especies, hábitats, avistamientos y XML.

Especies: permite listar, añadir, modificar, eliminar y buscar especies mediante formularios y tablas.

Hábitats: permite listar, añadir, modificar, eliminar y buscar hábitats mediante la interfaz gráfica.

Avistamientos: permite listar, añadir, modificar, eliminar y buscar avistamientos, manteniendo las comprobaciones de relación con especies y hábitats.

XML: permite exportar los datos y leer los diferentes ficheros XML desde la interfaz.

Al ejecutar Main.java aparece el menú principal de la aplicación.

El menú principal permite acceder a la gestión de especies, hábitats, avistamientos y XML.

Gestionar especies: permite listar, añadir, modificar, eliminar y buscar especies.

Gestionar hábitats: permite listar, añadir, modificar, eliminar y buscar hábitats.

Gestionar avistamientos: permite listar, añadir, modificar, eliminar y buscar avistamientos.

Gestionar XML: permite exportar los datos a XML y leer los ficheros XML generados.

Salir: termina la aplicación.

13. Puesta en marcha

Para ejecutar el proyecto es necesario disponer de Java y de IntelliJ IDEA. Se debe abrir el proyecto como un proyecto Java y comprobar que las dependencias necesarias de XStream están añadidas al proyecto.

Los ficheros binarios pueden generarse inicialmente ejecutando las clases EscribirEspecies, EscribirHabitats y EscribirAvistamientos. Estas clases crean los datos iniciales de prueba. Una vez creados los ficheros, la aplicación se ejecuta desde Main.java.

Las dependencias de XStream utilizadas por el proyecto deben mantenerse disponibles dentro del proyecto para que la aplicación pueda ejecutarse correctamente.

14. Git y control de versiones

El proyecto se ha mantenido mediante un repositorio Git. Durante el desarrollo se han realizado commits atómicos y frecuentes, identificando mediante el mensaje del commit los cambios realizados.

También se utiliza un fichero .gitignore para evitar incluir archivos generados por el IDE o archivos de compilación que no forman parte del código fuente.

15. Conclusiones

La incorporación de la interfaz gráfica amplía la aplicación y permite utilizar las operaciones principales de una forma más visual e intuitiva, manteniendo la gestión de los datos mediante los mismos ficheros y clases.

El proyecto permite aplicar los conceptos de acceso a datos trabajados durante la unidad, especialmente la utilización de ficheros binarios, serialización de objetos, acceso secuencial y generación y lectura de documentos XML. La incorporación de una interfaz gráfica mejora además la interacción del usuario con la aplicación.

La estructura separa las clases de modelo, las operaciones sobre ficheros, la gestión XML y el menú principal, facilitando la organización y mantenimiento de la aplicación. Además, las comprobaciones realizadas entre especies, hábitats y avistamientos permiten mantener la coherencia de la información almacenada. La interfaz gráfica actúa como capa de presentación y reutiliza la lógica existente.

16. Glosario

GUI: interfaz gráfica de usuario que permite interactuar con la aplicación mediante elementos visuales.

Swing: biblioteca gráfica de Java utilizada para desarrollar la interfaz de usuario.

Serializable: mecanismo de Java que permite convertir un objeto en una secuencia de bytes para almacenarlo o transmitirlo.

XML: formato de texto utilizado para representar y almacenar datos de forma estructurada.

XStream: librería Java utilizada para convertir objetos Java a XML y XML a objetos Java.

CRUD: conjunto de operaciones de crear, leer, actualizar y eliminar registros.

IDE: entorno integrado de desarrollo utilizado para escribir, ejecutar y depurar el código.